```
import torch
import torch.nn as nn
import torchvision
import torchvision.transforms as transforms
from torchvision.utils import save_image

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```
latent_size = 64
image_size = 28 * 28
batch_size = 128
num_epochs = 50
lr = 0.0002
```

```
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])

dataset = torchvision.datasets.MNIST(
    root='./data', train=True, transform=transform, download=True)

dataloader = torch.utils.data.DataLoader(
    dataset, batch_size=batch_size, shuffle=True)
```

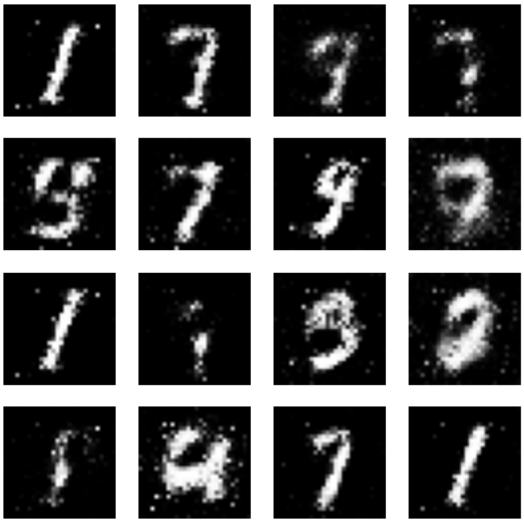
```
class Generator(nn.Module):
    def __init__(self):
        super().__init__()
        self.model = nn.Sequential(
            nn.Linear(latent_size, 128),
            nn.ReLU(True),
            nn.Linear(128, 256),
            nn.ReLU(True),
            nn.Linear(256, image_size),
            nn.Tanh()
        )

    def forward(self, z):
        out = self.model(z)
        return out.view(-1, 1, 28, 28)
```

```
class Discriminator(nn.Module):
    def __init__(self):
        super().__init__()
        self.model = nn.Sequential(
            nn.Linear(image_size, 256),
            nn.LeakyReLU(0.2),
            nn.Linear(256, 128),
            nn.LeakyReLU(0.2),
            nn.Linear(128, 1),
            nn.Sigmoid()
        )

    def forward(self, x):
        x = x.view(x.size(0), -1)
        return self.model(x)
```

```
G = Generator().to(device)
D = Discriminator().to(device)
```

```
criterion = nn.BCELoss()
d_optimizer = torch.optim.Adam(D.parameters(), lr=lr)
g_optimizer = torch.optim.Adam(G.parameters(), lr=lr)

for epoch in range(num_epochs):
    for i, (images, _) in enumerate(dataloader):

        images = images.to(device)
        batch_size_curr = images.size(0)

        real_labels = torch.ones(batch_size_curr, 1).to(device)
        fake_labels = torch.zeros(batch_size_curr, 1).to(device)

        # Train Discriminator
        outputs = D(images)
        d_loss_real = criterion(outputs, real_labels)
```

```
        z = torch.randn(batch_size_curr, latent_size).to(device)
        fake_images = G(z)
        outputs = D(fake_images.detach())
        d_loss_fake = criterion(outputs, fake_labels)

        d_loss = d_loss_real + d_loss_fake

        d_optimizer.zero_grad()
        d_loss.backward()
        d_optimizer.step()

        # Train Generator
        z = torch.randn(batch_size_curr, latent_size).to(device)
        fake_images = G(z)
        outputs = D(fake_images)

        g_loss = criterion(outputs, real_labels)

        g_optimizer.zero_grad()
        g_loss.backward()
        g_optimizer.step()

        if i % 200 == 0:
            print(f"Epoch [{epoch}/{num_epochs}] Step [{i}/{len(dataloader)}] "
                  f"D Loss: {d_loss.item():.4f}, G Loss: {g_loss.item():.4f}")

    # Save images
    save_image(fake_images[:25],
               f"mnist_fake_epoch_{epoch}.png",
               nrow=5,
               normalize=True)
```

```
Epoch [48/50] Step [400/469] D Loss: 0.5332, G Loss: 3.3334
Epoch [49/50] Step [0/469] D Loss: 0.4965, G Loss: 3.6133
Epoch [49/50] Step [200/469] D Loss: 0.4653, G Loss: 4.0994
Epoch [49/50] Step [400/469] D Loss: 0.6871, G Loss: 3.3384
```

```
save_image(fake_images[:25].view(-1,1,28,28),
           f"mnist_fake_epoch_{epoch}.png",
           nrow=5,
           normalize=True)

import matplotlib.pyplot as plt

fake = fake_images[:16].view(-1,28,28).detach().cpu()

fig, axes = plt.subplots(4,4, figsize=(5,5))
for i, ax in enumerate(axes.flatten()):
    ax.imshow(fake[i], cmap='gray')
    ax.axis('off')

plt.show()
```